

PROYECTO INTERMODULAR 2

SPRINT 2 – INFORME DE GESTION DE SCRUM

Miriam Mendieta Racedo.-
2DAMS
25/10/2026

SPRINT 2 – INFORME DE GESTIÓN DE SCRUM

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 25 de octubre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---------- **1**
EVOLUCIÓN DEL TRABAJO Y ESTADO DE HISTORIAS DE USUARIO (SPRINT 1) ---------- **4**
A. Historias de Usuario Completadas ---------- 4
B. Dificultades / Impedimentos Técnicos Solventados ---------- 11
C. Tareas No Finalizadas Traspasadas al Sprint 2 ---------- 11
D. Objetivo del Sprint 3 (Sprint Goal). ---------- 12
BIBLIOGRAFÍA ---------- **13**

INTRODUCCIÓN

Al iniciar este proyecto, el mayor desafío técnico no fue la sintaxis del lenguaje, sino tener que **aprender Spring Boot al mismo tiempo que desarrollaba el primer sprint**. Cada error me obligaba a detenerme, investigar, probar, corregir… y volver a empezar. Esa combinación de descubrimiento y desarrollo me hizo llegar justísima a la entrega, con la sensación de haber sobrevivido más que de haber trabajado con seguridad.

Pero algo mucho más revelador ocurrió: mi cabeza hizo *click*. Pasé de pensar en **ejercicios aislados** a enfrentarme a un **sistema integral**, donde cada pieza es una ficha de dominó capaz de derribar a las demás.

En los ejercicios de cada lenguaje que aprendemos en la formación, el código vive en entornos controlados, con datos volátiles en memoria y sin dependencias externas. Sin embargo, esta app exige la convergencia de múltiples capas reales:

- **Persistencia:** Gestión del estado mediante una base de datos.
- **Presentación:** Renderizado dinámico del lado del servidor con Thymeleaf, HTML y CSS.
- **Control de Flujo:** Desacoplamiento de la lógica de negocio del protocolo HTTP.

Modificar una vista podía romper el sistema entero. Y la única forma de comprobar si todo seguía funcionando era abrir el navegador, rellenar formularios una y otra vez, esperando que nada hubiera caído por el camino. Era un flujo agotador, frágil y poco profesional.

Fue en ese caos donde entendí la verdad que marca este Sprint 2: si en cada paso **no refactorizo, el sistema se vuelve rígido y repetitivo; si no testeo, el sistema se vuelve impredecible.**

En síntesis, construir software no es solo escribir código, también es metodología, refactorización, testing y validación para evitar acoplamientos y prevenir fallos en cadena.

Estrategia de Mitigación: Refactoring y Testing Automatizado

A. *Arquitectura y Refactoring (Desacoplamiento)*

El primer paso consiste en aislar las responsabilidades aplicando el principio de responsabilidad única (SRP):

- ***Separación por Capas:*** Se dividió estrictamente el proyecto en "Controllers" (Thymeleaf/HTTP), "Services" (Lógica de negocio pura) y "Repositories" (Acceso a datos). Esto ya esta separado gracias a la implementación del proyecto a traves de Spring boot.

- ***Inyección de Dependencias:*** eliminación de instanciación directa de objetos (uso de "new") en favor de la inyección por constructor nativa de Spring Boot. Esto rompe el acoplamiento y permite la sustitución de componentes. Toca implementarlo a través de investigación y estudio del tema.

B. *Implementación del Testing Automatizado*

Con el código desacoplado, se implementa en una suite de pruebas automatizadas enfocada en la ***Pirámide de Pruebas*** , priorizando el retorno de inversión de tiempo y esfuerzo:

- ***Pruebas*** Unitarias Aisladas (JUnit 5 + Mockito): Se testea la lógica crítica (como el flujo de login o validaciones) de forma 100% aislada. Se utilizan "Mocks" para simular el comportamiento de la base de datos, garantizando que los tests se ejecuten sin depender de infraestructura externa.

- ***Flujo*** de Trabajo Paralelo: Cada funcionalidad nace acompañada de su test. No después, no "cuando haya tiempo". Justo después de implementarla. Así ninguna pieza queda expuesta y el sistema crece blindado desde dentro.

Con toda esta estrategia se pretende tener una mejora significativa en la gestión del tiempo y la calidad del código a presentar ya que combina una red de seguridad mediante tests, refactoring continuo para evitar deuda técnica y un desarrollo más ágil y eficiente que elimina el testing manual.

Reducción del Alcance del MVP

La decisión de disminuir nuevamente el alcance del MVP responde a una necesidad estratégica de garantizar la entrega de un producto **funcional, estable y demostrable** dentro del plazo límite del **22 de noviembre**. Durante el desarrollo inicial se identificó que el alcance previsto en la planificación original excedía la capacidad real de implementación dentro del tiempo disponible, especialmente considerando la complejidad de algunos módulos avanzados (gestión documental, alertas, OCR, backups, panel de activos completo, etc.).

Para evitar riesgos de retraso y asegurar un resultado optimo, se optó por **redefinir el MVP** centrándolo únicamente en los elementos esenciales que permiten validar el funcionamiento del sistema y demostrar su utilidad real:

- Gestión básica de activos digitales (CRUD).
- Registro de movimientos para trazabilidad.
- Roles y acceso diferenciado (usuario / administrador/Auditor).
- Gestión de usuario(Admistrador)
- Navegación mínima y estable.

Esta reducción garantiza que el producto entregado:

1. **Cumpla los requisitos mínimos funcionales** del proyecto.
2. **Sea demostrable** en vídeo sin errores críticos.
3. **Mantenga coherencia técnica** y una arquitectura escalable.
4. **Permita añadir módulos adicionales posteriormente** sin necesidad de rehacer la base del sistema.

Además, esta decisión se alinea con las buenas prácticas de desarrollo ágil: un MVP debe centrarse en las funcionalidades que aportan valor inmediato y permiten validar el concepto, dejando las mejoras y módulos complementarios para iteraciones posteriores.

En resumen, la reducción del alcance del MVP no implica una pérdida de calidad, sino una **optimización estratégica** para asegurar la entrega en fecha, evitar sobrecarga de trabajo y garantizar que el producto final sea **funcional, estable y ampliable**. Una vez entregado el MVP, se podrá continuará con el desarrollo progresivo de los módulos restantes, siguiendo una planificación más realista y sostenible.

Evolución del Trabajo y Estado de Historias de Usuario (Sprint 1)

A. Historias de Usuario Completadas

HU 01.1B – Acceso al Sistema: Logout

Descripción

Permite al usuario cerrar sesión de forma segura, invalidando la sesión activa y evitando el acceso a zonas privadas mediante navegación Atrás. Tras el logout, el sistema redirige al usuario a la pantalla de inicio de sesión.

Tareas técnicas

- Implementar endpoint de logout en Spring Security.
- Invalidar sesión correctamente.
- Configurar redirección al login tras cerrar sesión.
- Bloquear navegación atrás después del logout.
- Probar cierre de sesión desde distintas vistas.

Criterios de aceptación

1. **El usuario puede cerrar sesión correctamente**
   - Al hacer clic en “Cerrar sesión”, el sistema ejecuta el endpoint de logout configurado en Spring Security.
   - La sesión activa se invalida inmediatamente.
   - La autenticación del usuario se limpia por completo.
2. **Redirección obligatoria al login**
   - Tras el logout, el sistema redirige automáticamente a la pantalla de inicio de sesión.
   - No se muestra ninguna vista privada después del logout.

3. **Navegación atrás bloqueada**
   Si el usuario intenta volver atrás con el navegador después del logout:
   - No puede acceder a vistas privadas.
   - El sistema redirige a /login.
   - Las páginas privadas no se muestran desde caché.

4. **Acceso a zonas privadas prohibido**
   Cualquier intento de acceder a rutas privadas sin sesión activa:
   - Debe ser bloqueado por Spring Security.
   - Debe redirigir a /login.

5. **Pruebas funcionales**
   - Cerrar sesión desde cualquier vista privada funciona correctamente.
   - Intentar navegar atrás después del logout muestra login.
   - Intentar acceder manualmente a una URL privada después del logout muestra login.
   - La sesión no se mantiene en ningún caso tras el logout.

6. **Comportamiento consistente en todo el sistema**
   - El botón “Salir” que se utiliza para cerrar sesión está disponible en todas las vistas.
   - El comportamiento del logout es idéntico independientemente de la vista desde la que se ejecuta.

HU 03.2 – Visualización Segura de Contraseñas

Descripción

Mejora la usabilidad y seguridad de los formularios permitiendo mostrar u ocultar la contraseña mediante un botón de visualización, manteniendo el campo enmascarado por defecto.

Tareas técnicas

- Implementar campo de contraseña con enmascarado.
- Añadir botón “ver/ocultar contraseña”.
- Asegurar que el toggle no afecta la seguridad del formulario.
- Probar en login y recuperación de contraseña.

Criterios de aceptación

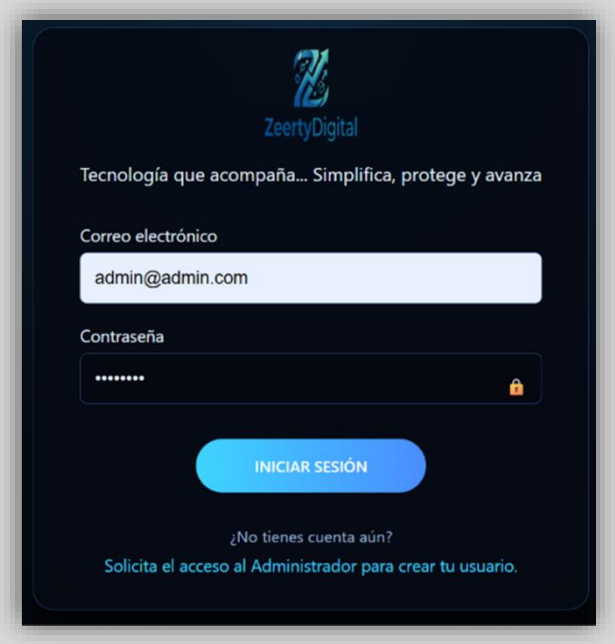
1. **El campo de contraseña está enmascarado por defecto**
    - El input debe mostrarse inicialmente como type="password".
    - El usuario no puede ver la contraseña hasta que pulse el botón de visualización.

2. **Existe un toggle para mostrar/ocultar la contraseña**
    - El toggle está claramente visible dentro del campo de contraseña.
    - El icono del toggle cambia según el estado: mostrar/ocultar.

3. **El toggle no compromete la seguridad**
    - El cambio de tipo del input **no debe afectar** la validación del formulario.
    - No debe permitir copiar la contraseña automáticamente.
    - No debe almacenar la contraseña en ningún lugar adicional.
    - No debe romper políticas de seguridad del sistema.

4. **No se permite dejar la contraseña visible tras enviar el formulario**
    - Al enviar el formulario, el campo debe volver a estar enmascarado.
    - No debe quedar visible en caso de error de validación.

5. **Pruebas funcionales**
    - Mostrar contraseña → funciona correctamente.
    - Ocultar contraseña → funciona correctamente.
    - El icono cambia según el estado.
    - El formulario sigue validando la contraseña igual que antes.
    - No se producen errores visuales ni de layout.





HU 01.X – Gestión de Usuarios por el Administrador

Descripción

El administrador puede dar de alta nuevos usuarios, editar sus roles y activar o desactivar cuentas. Esta funcionalidad permite controlar quién accede al sistema y con qué permisos.

Tareas técnicas

- Crear formulario de alta de usuario.
- Validar correo duplicado.
- Implementar endpoint para crear usuarios.
- Añadir selector de roles.
- Implementar activación/desactivación de usuarios.
- Proteger vista solo para ROLE_ADMIN.
- Probar alta, edición y desactivación.

Criterios de Aceptación

1. **Acceso exclusivo para ROLE_ADMIN**
    - Solo usuarios con **ROLE_ADMIN** pueden acceder a la vista de gestión de usuarios.
    - No se permite acceder mediante URL directa sin ROLE_ADMIN.

2. **Alta de nuevos usuarios**
    - El administrador puede abrir el formulario de alta de usuario.
    - El formulario contiene campos obligatorios: nombre, correo, rol, estado.
    - Si el correo ya existe en el sistema →se muestra error y no se crea el usuario.
    - Si los datos son válidos → el usuario se crea correctamente y aparece en la lista.

3. **Selector de roles funcional**
    - El administrador puede asignar un rol al usuario desde un selector (dropdown).
    - Los roles disponibles se muestran correctamente.
    - El rol seleccionado se guarda y afecta a los permisos del usuario.

4. **Edición de usuarios**
    - El administrador puede modificar datos básicos del usuario (nombre, rol, estado).
    - Si se intenta asignar un correo duplicado → se bloquea la edición y se muestra error.
    - Los cambios válidos se guardan correctamente y se reflejan en la lista.

5. **Activación y desactivación de usuarios**
    - El administrador puede activar o desactivar usuarios desde la vista de gestión.
    - Un usuario desactivado **no puede iniciar sesión**.
    - El estado del usuario se actualiza correctamente en la base de datos.
    - La interfaz refleja el estado actualizado (activo/inactivo).

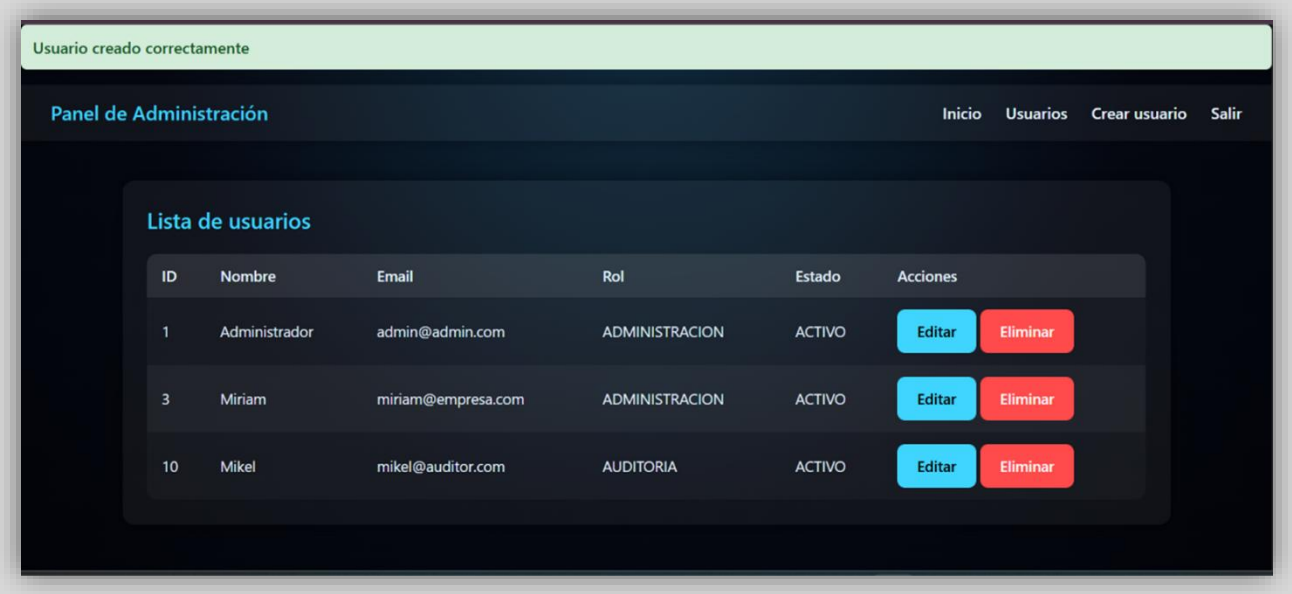
6. **Seguridad del módulo**
    - Todas las operaciones (alta, edición, activación/desactivación) están protegidas por Spring Security.
    - No se permite realizar acciones administrativas mediante llamadas directas sin ROLE_ADMIN.
    - No se filtra información sensible en errores o respuestas.

7. **Pruebas**
    - Alta de usuario con datos válidos → OK.
    - Alta de usuario con correo duplicado → ERROR.
    - Edición de usuario con datos válidos → OK.

- Edición con correo duplicado → ERROR.
- Activación/desactivación → OK.
- Intento de acceso sin ROLE_ADMIN → bloqueado.
- Validación de que un usuario desactivado no puede iniciar sesión. → esta prueba ha fallado se crea una nueva HU para poder solucionarla.

8. Cumple la Definition of Done

- Código revisado.
- Sin errores visibles.
- Documentación mínima actualizada.
- HU marcada como completada en Trello.



B. Dificultades / Impedimentos Técnicos Solventados

Tipo de Impedimento	Descripción del Problema	Impacto en el Sprint	Estrategia de Solución	Medidas Preventivas / Lecciones Aprendidas
Reorganización del MVP				

c. Tareas No Finalizadas Traspasadas al Sprint 2

D. Objetivo del Sprint 3 (Sprint Goal).

Historias de Usuario seleccionadas del Product Backlog para este Sprint (SprintBacklog)

ID HU	Nombre	Descripción breve	Estimación (puntos)

Desglose inicial de tareas técnicas

Resumen del Sprint Goal

ANEXO

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.